NATIONAL UNIVERSITY OF HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY



Microprocessors-Microcontrollers (CO3010)

LAB REPORT

Instructor: Phan Văn Sỹ

Class CC03

Full Name	Student ID
TRƯƠNG ANH KHOA	2452566

Table of Contents

1 Exercise 1: 2-LED Alternating Blinking **3**

2 Exercise 2: Simple Traffic Light **4**

3 Exercise 3: 4-Way Traffic Light **5**

4 Exercise 4: 7-Segment LED Display Function **6**

5 Exercise 5: Traffic Light with Countdown Timer **8**

6 Exercise 6: 12-LED Analog Clock Test **9**

7 Exercise 7, 8 & 9: Clock Control Helper Functions **11**

8 Exercise 10: Full Analog Clock Implementation **12**

1 Exercise 1: 2-LED Alternating Blinking

Objective: Switch the status of a Red LED and a Yellow LED every 2 seconds.

Methodology: Pins PA5 and PA6 on the STM32F103C6 are configured as GPIO Output. In the Proteus simulation, the LEDs are connected in an active-low configuration (anodes connected to +3.3V and cathodes connected to the MCU pins). In the main `while(1)` loop, the `HAL_GPIO_WritePin` function is used to toggle the pins between SET (Logic 1 = OFF) and RESET (Logic 0 = ON) with a 2000ms delay.

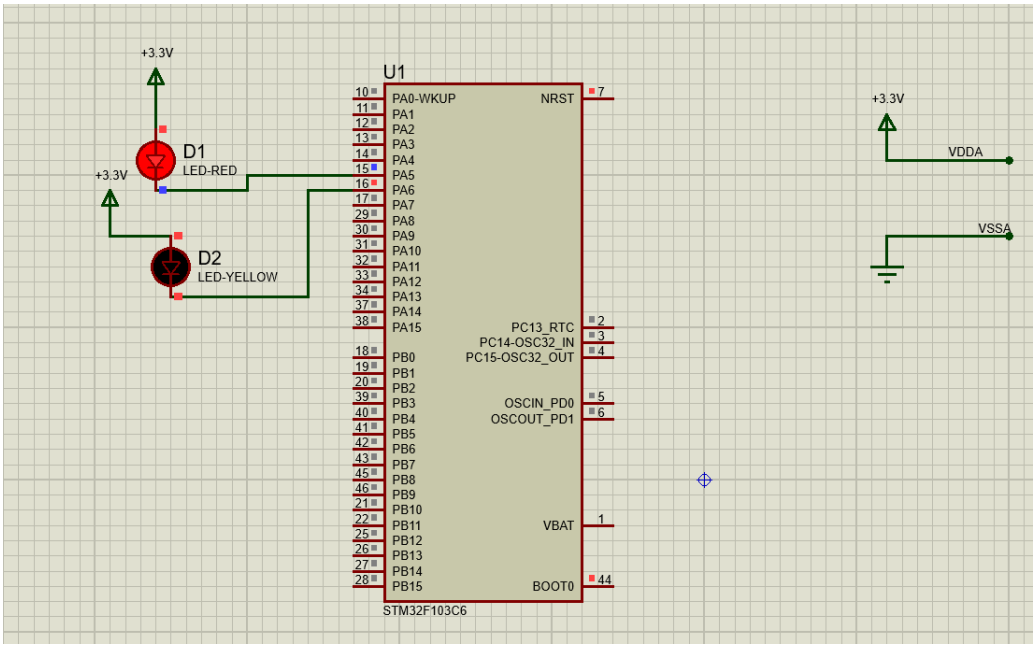


Figure 1: Proteus Schematic for Exercise 1

Source Code:

```
while (1) {
    HAL_GPIO_WritePin(LED_RED_GPIO_Port, LED_RED_Pin, GPIO_PIN_RESET);
    HAL_GPIO_WritePin(LED_YELLOW_GPIO_Port, LED_YELLOW_Pin,
       GPIO_PIN_SET);
    HAL_Delay(2000);

    HAL_GPIO_WritePin(LED_RED_GPIO_Port, LED_RED_Pin, GPIO_PIN_SET);
    HAL_GPIO_WritePin(LED_YELLOW_GPIO_Port, LED_YELLOW_Pin,
       GPIO_PIN_RESET);
    HAL_Delay(2000);
}
```

2 Exercise 2: Simple Traffic Light

Objective: Simulate a traffic light behavior with Red (5s), Green (3s), and Yellow (2s).

Methodology: A third LED (Green) is added and connected to pin PA7, which is configured as GPIO Output. The program executes a sequential logic representing three traffic states. Appropriate delays (5000ms, 3000ms, 2000ms) are inserted after setting the respective LED pin to RESET and the others to SET.

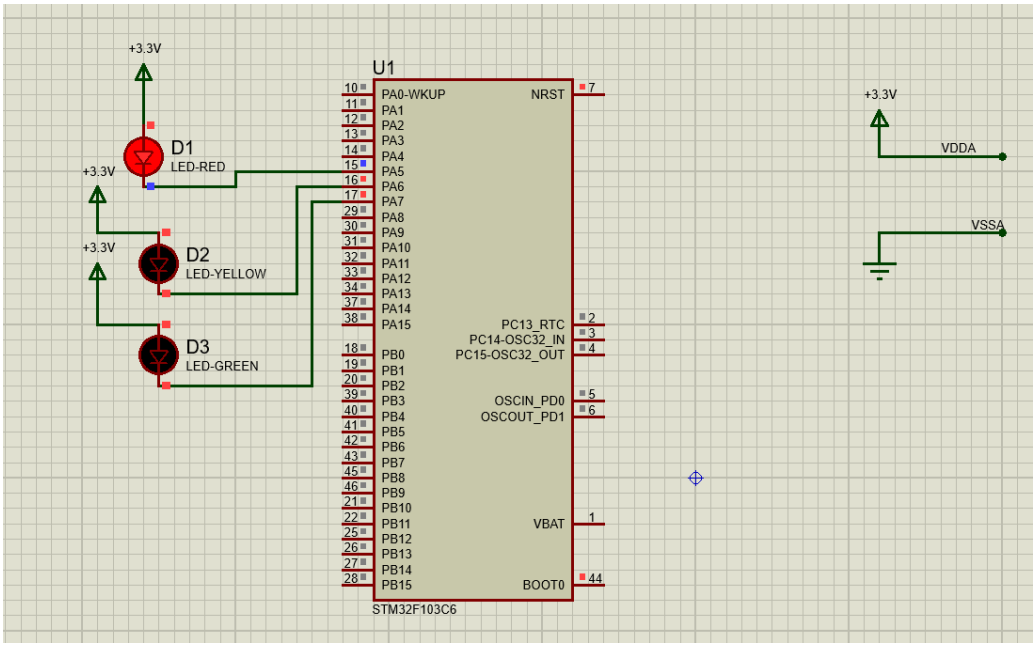


Figure 2: Proteus Schematic for Exercise 2

Source Code:

```
while (1) {
    HAL_GPIO_WritePin(LED_RED_GPIO_Port, LED_RED_Pin, GPIO_PIN_RESET);
    HAL_GPIO_WritePin(LED_YELLOW_GPIO_Port, LED_YELLOW_Pin,
       GPIO_PIN_SET);
    HAL_GPIO_WritePin(LED_GREEN_GPIO_Port, LED_GREEN_Pin, GPIO_PIN_SET
       );
    HAL_Delay(5000);

    HAL_GPIO_WritePin(LED_RED_GPIO_Port, LED_RED_Pin, GPIO_PIN_SET);
    HAL_GPIO_WritePin(LED_YELLOW_GPIO_Port, LED_YELLOW_Pin,
       GPIO_PIN_RESET);
    HAL_GPIO_WritePin(LED_GREEN_GPIO_Port, LED_GREEN_Pin, GPIO_PIN_SET
       );
    HAL_Delay(2000);

    HAL_GPIO_WritePin(LED_RED_GPIO_Port, LED_RED_Pin, GPIO_PIN_SET);
```

```
    HAL_GPIO_WritePin(LED_YELLOW_GPIO_Port, LED_YELLOW_Pin,
        GPIO_PIN_SET);
    HAL_GPIO_WritePin(LED_GREEN_GPIO_Port, LED_GREEN_Pin,
        GPIO_PIN_RESET);
    HAL_Delay(3000);
}
```

3 Exercise 3: 4-Way Traffic Light

Objective: Extend the traffic light to a 4-way intersection using 12 LEDs.

Methodology: Three additional pins (PA8, PA9, PA10) are configured as outputs to control the East-West axis. The North-South axis continues to use PA5, PA6, PA7. In Proteus, 12 LEDs are arranged in a cross junction. To optimize the microcontroller's GPIO usage, opposite directions (North-South and East-West) are wired in parallel to share the same control signals. The code implements a 4-state sequence to ensure safe traffic flow.

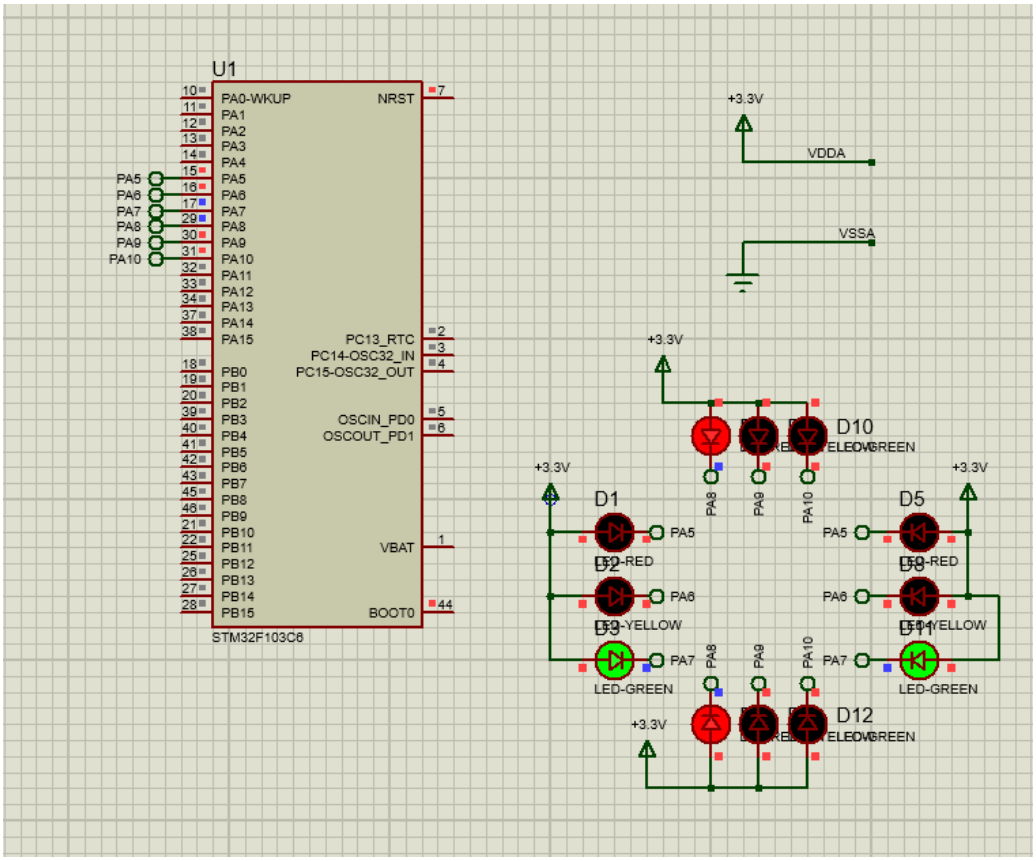


Figure 3: Proteus Schematic for Exercise 3

Source Code:

```
while (1) {
```

```
    // State 1: N-S Green, E-W Red
    HAL_GPIO_WritePin(GPIOA, LED_RED_Pin|LED_YELLOW_Pin, GPIO_PIN_SET)
        ;
    HAL_GPIO_WritePin(GPIOA, LED_GREEN_Pin, GPIO_PIN_RESET);
    HAL_GPIO_WritePin(GPIOA, LED_RED_2_Pin, GPIO_PIN_RESET);
    HAL_GPIO_WritePin(GPIOA, LED_YELLOW_2_Pin|LED_GREEN_2_Pin,
        GPIO_PIN_SET);
    HAL_Delay(3000);

    // State 2: N-S Yellow, E-W Red
    HAL_GPIO_WritePin(GPIOA, LED_GREEN_Pin, GPIO_PIN_SET);
    HAL_GPIO_WritePin(GPIOA, LED_YELLOW_Pin, GPIO_PIN_RESET);
    HAL_Delay(2000);

    // State 3: N-S Red, E-W Green
    HAL_GPIO_WritePin(GPIOA, LED_YELLOW_Pin, GPIO_PIN_SET);
    HAL_GPIO_WritePin(GPIOA, LED_RED_Pin, GPIO_PIN_RESET);
    HAL_GPIO_WritePin(GPIOA, LED_RED_2_Pin, GPIO_PIN_SET);
    HAL_GPIO_WritePin(GPIOA, LED_GREEN_2_Pin, GPIO_PIN_RESET);
    HAL_Delay(3000);

    // State 4: N-S Red, E-W Yellow
    HAL_GPIO_WritePin(GPIOA, LED_GREEN_2_Pin, GPIO_PIN_SET);
    HAL_GPIO_WritePin(GPIOA, LED_YELLOW_2_Pin, GPIO_PIN_RESET);
    HAL_Delay(2000);
}
```

4 Exercise 4: 7-Segment LED Display Function

Objective: Implement a function to decode and display numbers from 0 to 9 on a 7-segment LED (Common Anode type).

Methodology: Pins PB0 to PB6 are configured as GPIO Output. A common anode 7-segment display is connected to these pins. An array `seg7[10]` stores the hexadecimal values corresponding to the segments (a-g) for digits 0-9. Since it is a common anode display, a logic '0' turns a segment ON. The function uses bitwise shift operations `>>` and bitwise AND `&` to extract and output individual bit values to the respective pins.

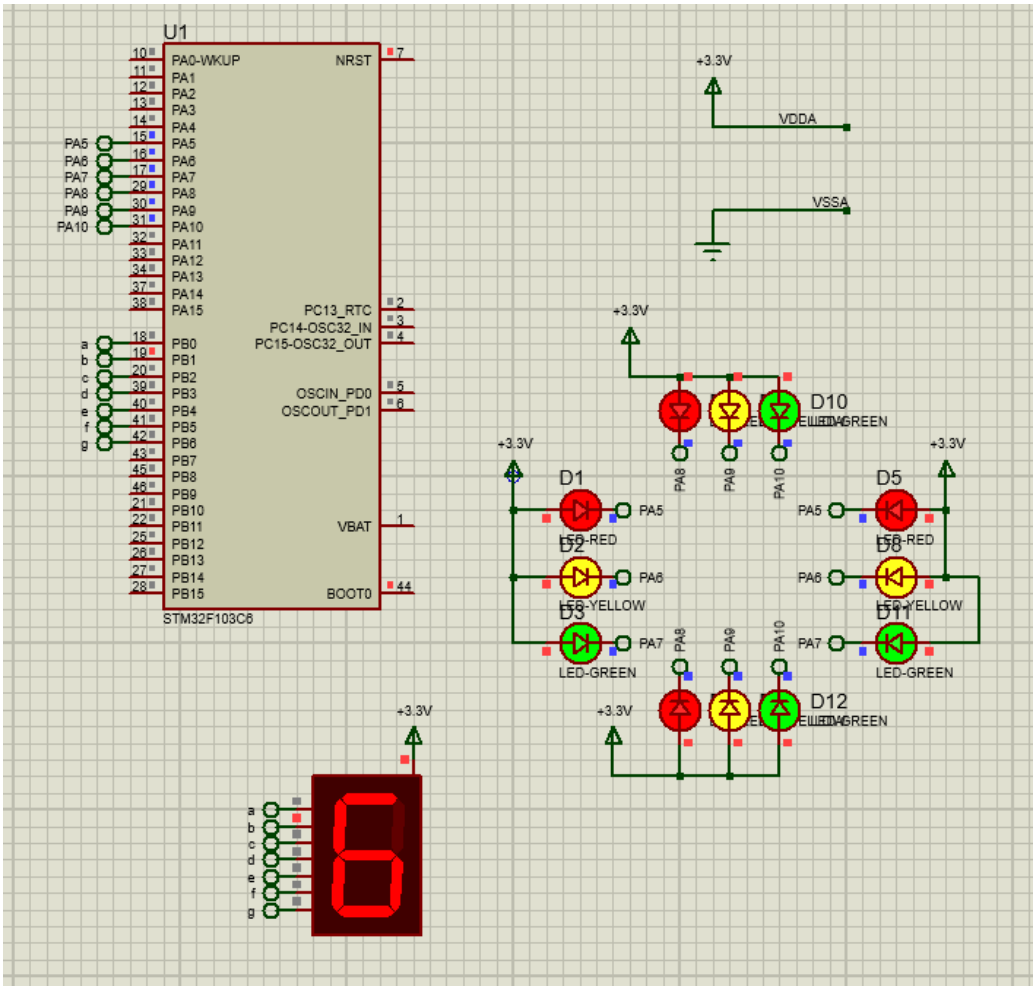


Figure 4: Proteus Schematic for Exercise 4

Source Code:

```
void display7SEG(int num) {
    uint8_t seg7[10] = {0x40, 0x79, 0x24, 0x30, 0x19, 0x12, 0x02, 0x78
        , 0x00, 0x10};

    if (num < 0 || num > 9) return;

    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_0, (seg7[num] >> 0) & 0x01); //
        a
    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_1, (seg7[num] >> 1) & 0x01); //
        b
    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_2, (seg7[num] >> 2) & 0x01); //
        c
    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_3, (seg7[num] >> 3) & 0x01); //
        d
    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_4, (seg7[num] >> 4) & 0x01); //
        e
```

```
    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_5, (seg7[num] >> 5) & 0x01); // f
    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_6, (seg7[num] >> 6) & 0x01); // g
}
```

5 Exercise 5: Traffic Light with Countdown Timer

Objective: Integrate the 7-segment LED with the 4-way traffic light to display countdown values iteratively for each state sequence.

Methodology: The static `HAL_Delay()` functions in the traffic light routine are replaced by `for` loops. Each loop decrements a counter variable `i` and calls `display7SEG(i)`, followed by a 1-second delay. This approach simultaneously manages the traffic light phases and updates the 7-segment countdown timer continuously.

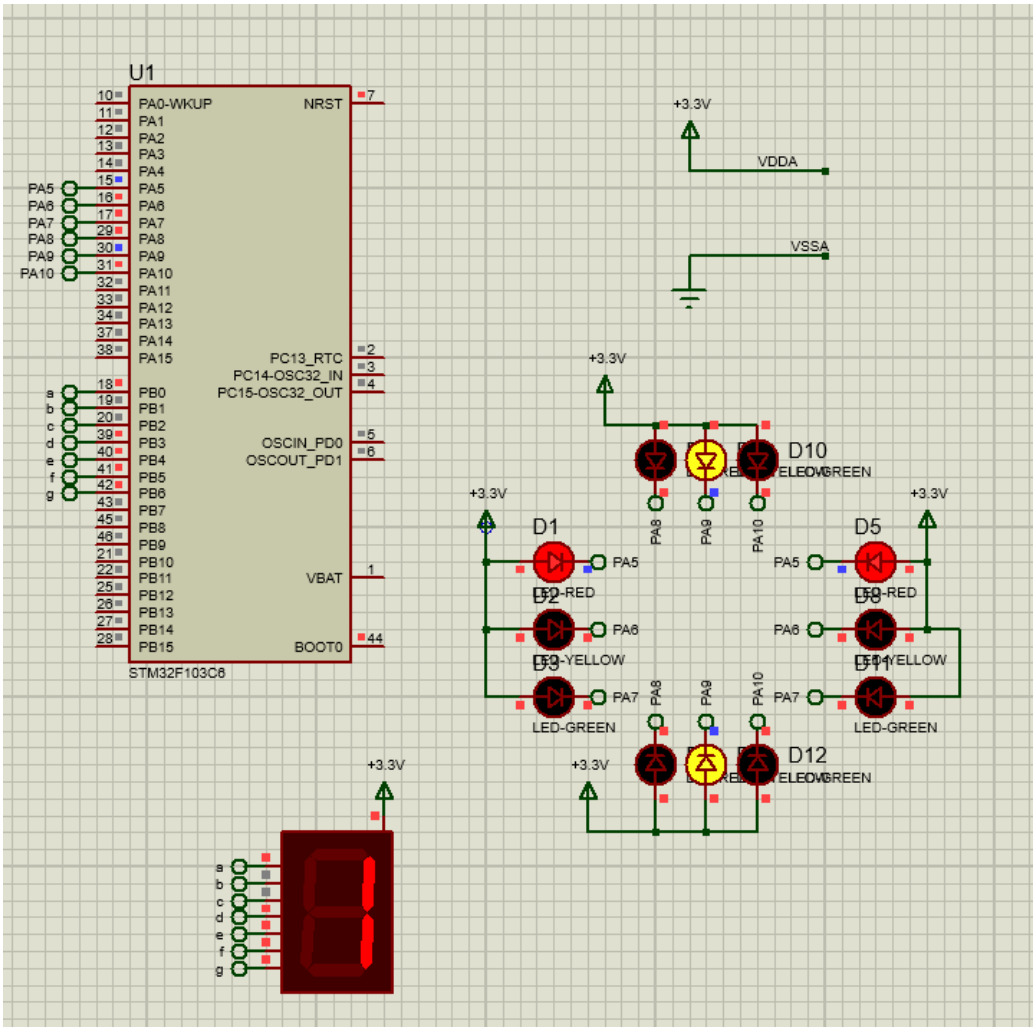


Figure 5: Proteus Schematic for Exercise 5

Source Code:

```
while (1) {
    HAL_GPIO_WritePin(GPIOA, LED_RED_Pin|LED_YELLOW_Pin, GPIO_PIN_SET)
       ;
    HAL_GPIO_WritePin(GPIOA, LED_GREEN_Pin, GPIO_PIN_RESET);
    HAL_GPIO_WritePin(GPIOA, LED_RED_2_Pin, GPIO_PIN_RESET);
    HAL_GPIO_WritePin(GPIOA, LED_YELLOW_2_Pin|LED_GREEN_2_Pin,
       GPIO_PIN_SET);
    for (int i = 5; i >= 0; i -= 1){
        display7SEG(i);
        HAL_Delay(1000);
    }

    HAL_GPIO_WritePin(GPIOA, LED_GREEN_Pin, GPIO_PIN_SET);
    HAL_GPIO_WritePin(GPIOA, LED_YELLOW_Pin, GPIO_PIN_RESET);
    for (int i = 2; i >= 0; i -= 1){
        display7SEG(i);
        HAL_Delay(1000);
    }

    HAL_GPIO_WritePin(GPIOA, LED_YELLOW_Pin, GPIO_PIN_SET);
    HAL_GPIO_WritePin(GPIOA, LED_RED_Pin, GPIO_PIN_RESET);
    HAL_GPIO_WritePin(GPIOA, LED_RED_2_Pin, GPIO_PIN_SET);
    HAL_GPIO_WritePin(GPIOA, LED_GREEN_2_Pin, GPIO_PIN_RESET);
    for (int i = 5; i >= 0; i -= 1){
        display7SEG(i);
        HAL_Delay(1000);
    }

    HAL_GPIO_WritePin(GPIOA, LED_GREEN_2_Pin, GPIO_PIN_SET);
    HAL_GPIO_WritePin(GPIOA, LED_YELLOW_2_Pin, GPIO_PIN_RESET);
    for (int i = 2; i >= 0; i -= 1){
        display7SEG(i);
        HAL_Delay(1000);
    }
}
```

6 Exercise 6: 12-LED Analog Clock Test

Objective: Simulate an analog clock with 12 LEDs and test the hardware connection by turning on every LED sequentially.

Methodology: Pins PA4 to PA15 are configured as outputs. An array `clock_pins[12]`

is initialized to store these pin definitions. In the main loop, a nested `for` loop is utilized: the inner loop clears all LEDs, and the outer loop activates the LED at the current index, creating a rotating sequence effect with a 1000ms delay.

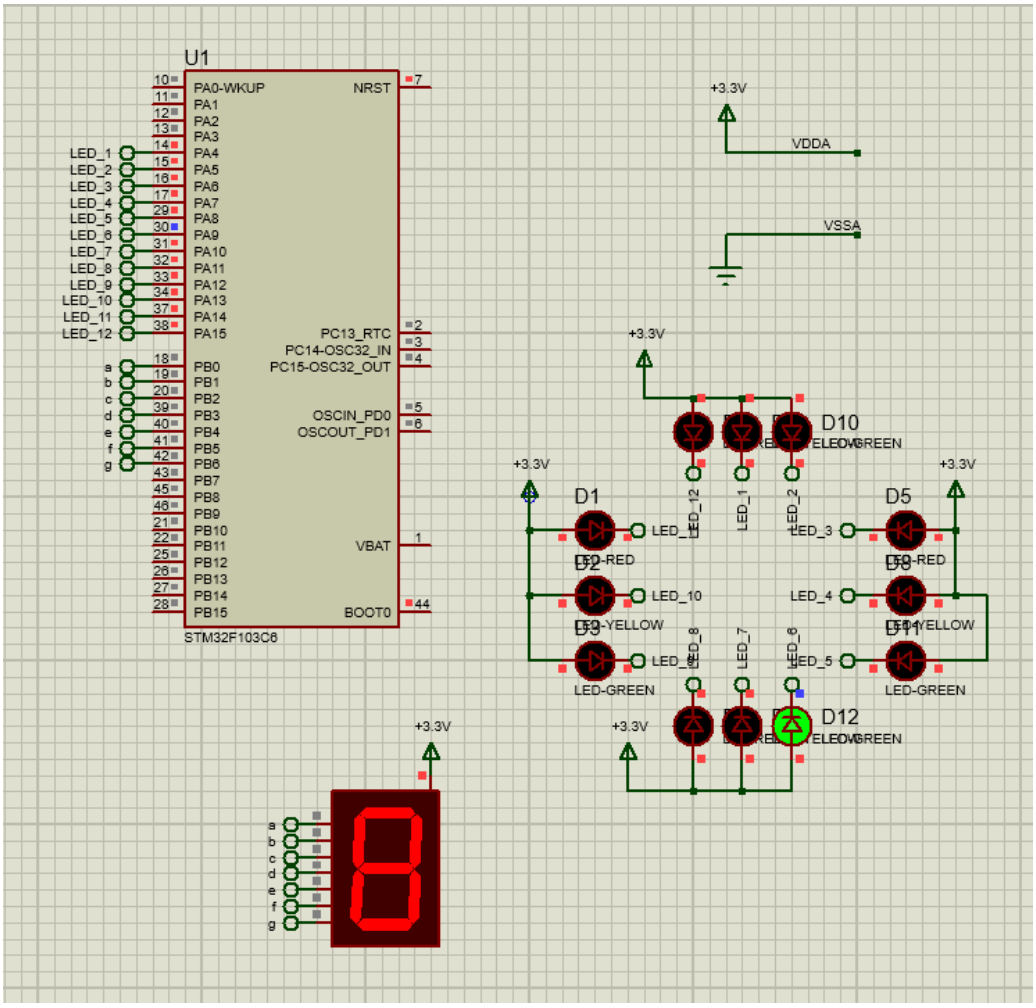


Figure 6: Proteus Schematic for Exercise 6

Source Code:

```
uint16_t clock_pins[12] = {LED_1_Pin, LED_2_Pin, LED_3_Pin, LED_4_Pin,
    LED_5_Pin, LED_6_Pin, LED_7_Pin, LED_8_Pin, LED_9_Pin, LED_10_Pin,
    LED_11_Pin, LED_12_Pin};

while (1) {
    for (int i = 0; i < 12; i++){
        for(int j = 0; j < 12; j++){
            HAL_GPIO_WritePin(GPIOA, clock_pins[j], GPIO_PIN_SET);
        }
        HAL_GPIO_WritePin(GPIOA, clock_pins[i], GPIO_PIN_RESET);
        HAL_Delay(1000);
```

```
    }
}
```

7 Exercise 7, 8 & 9: Clock Control Helper Functions

Objective: Implement modular functions to clear all LEDs, set a specific LED, and clear a specific LED on the clock face.

Methodology:

- `clearAllClock()`: Iterates through the `clock_pins` array and sets all pins to HIGH (OFF state).
- `setNumberOnClock(int num)`: Validates the input index (0-11) and sets the corresponding pin to LOW (ON state).
- `clearNumberOnClock(int num)`: Validates the input index and sets the corresponding pin to HIGH (OFF state).

Source Code:

```
uint16_t clock_pins[12] = {LED_1_Pin, LED_2_Pin, LED_3_Pin, LED_4_Pin,
    LED_5_Pin, LED_6_Pin, LED_7_Pin, LED_8_Pin, LED_9_Pin, LED_10_Pin,
    LED_11_Pin, LED_12_Pin};

// Exercise 7
void clearAllClock(){
    for (int i = 0; i < 12; i++){
        HAL_GPIO_WritePin(GPIOA, clock_pins[i], GPIO_PIN_SET);
    }
}

// Exercise 8
void setNumberOnClock(int num){
    if (num < 0 || num > 11) return;
    HAL_GPIO_WritePin(GPIOA, clock_pins[num], GPIO_PIN_RESET);
}

// Exercise 9
void clearNumberOnClock(int num){
    if (num < 0 || num > 11) return;
    HAL_GPIO_WritePin(GPIOA, clock_pins[num], GPIO_PIN_SET);
}
```

8 Exercise 10: Full Analog Clock Implementation

Objective: Integrate the helper functions to build a functional analog clock. At any given time, exactly 3 LEDs are turned on to represent the hour, minute, and second markers.

Methodology: Three variables (`hour`, `minute`, `second`) track the current time. Since the clock face only has 12 LEDs to represent 60 minutes/seconds, the minute and second values are divided by 5 to map correctly to the 0-11 index range. The program clears the clock face, sets the 3 hands, and increments the second counter with rollover conditions for minutes and hours. The delay is set to 50ms to accelerate the simulation process in Proteus.

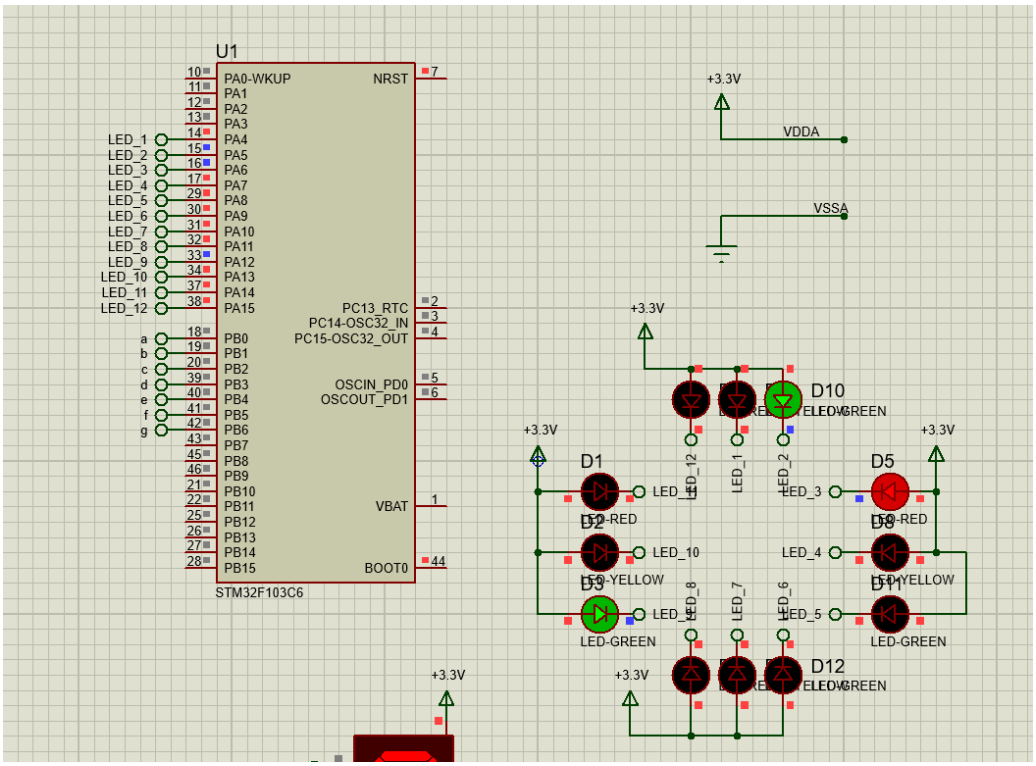


Figure 7: Proteus Schematic for Exercise 10

Source Code:

```
int hour = 0, minute = 0, second = 0;

while (1) {
    clearAllClock();
    setNumberOnClock(hour);
    setNumberOnClock(minute / 5);
    setNumberOnClock(second / 5);

    second++;
    if (second >= 60) {
        second = 0;
```

```
12          minute++;
13      }
14      if (minute >= 60) {
15          minute = 0;
16          hour++;
17      }
18      if (hour >= 12) {
19          hour = 0;
20      }
21      HAL_Delay(50);
22  }
```